

Relatório do trabalho prático 1 - Programação Paralela com OpenMP

Ana Laura Rizzo e Gislayne Garabini Damasceno

¹ Faculdade de Computação
Universidade Federal de Mato Grosso do Sul (UFMS) – Campo Grande – Brasil

{ana.rizzo, gislayne_garabini}@ufms.br

Resumo. *O presente trabalho trata-se da análise e a implementação do método iterativo de Jacobi para solução de sistemas lineares, de forma sequencial e paralela. Inicialmente, foi desenvolvido um algoritmo sequencial para a resolução do problema, seguido por uma versão paralela do problema utilizando múltiplas threads. A paralelização foi realizada no laço interno, responsável pelo cálculo das novas aproximações do vetor solução.*

1. Introdução

A resolução de sistemas lineares é um problema presente em várias áreas da computação, matemática aplicada e física. Ele resolve problemas práticos e importantes, por exemplo, problemas relacionados a tráfego de veículos, balanceamento de equações químicas, processamento de imagens e interpolação polinomial.

Com o aumento da complexidade computacional, tornou-se necessário utilizar técnicas avançadas, como o paralelismo, para reduzir o tempo de processamento e melhorar o desempenho dos algoritmos. Dessa forma utilizar ferramentas como OpenMP e arquiteturas com memória compartilhada é uma ótima alternativa. Para matrizes de grandes dimensões e porcentagem de coeficientes nulos, métodos iterativos tornam-se atraentes [Boccardo 2017].

O método iterativo de Jacobi é um algoritmo clássico para a resolução de sistemas lineares. Consiste em calcular sucessivas aproximações para o vetor solução até que atinja o critério de convergência. Então seja $x^{(0)}$ uma solução inicial para o sistema. Calcula o segundo valor $x^{(1)}$, da sequência x^k a partir do $x^{(0)}$ [Romanazzi 2020]. Uma característica do método, é que todos os elementos do vetor da próxima iteração podem ser calculados independentemente, utilizando apenas os valores dados pela iteração anterior. Isso torna o método de Jacobi ótimo para se paralelizar.

Neste trabalho foi utilizado a ferramenta OpenMP para paralelizar o algoritmo sequencial do método iterativo de Jacobi na linguagem C. O objetivo principal do trabalho foi analisar o impacto que o paralelismo trouxe no desempenho do algoritmo e observar métricas como o tempo de execução, o speedup e a eficiência dado uma quantidade diferente de threads.

2. Implementação

Foi desenvolvida na linguagem C, utilizando a API OpenMP para realizar a paralelização. Foram criadas duas versões do algoritmo:

- A versão sequencial.
- A versão paralela.

O sistema linear foi representado por uma matriz quadrada A , um vetor de termos independentes b e os vetores auxiliares x_{old} , x_{new} que são responsáveis por armazenar os valores da iteração anterior e da iteração atual, respectivamente.

2.1. Sequencial

Na versão sequencial, todas as operações são realizadas sequencialmente por uma única thread. O algoritmo percorre todas as linhas da matriz para calcular cada elemento do vetor solução.

Estrutura do código:

```

1  Inicializa x_old e x_new com zeros
2
3  Repete ate convergir:
4      Para cada linha i:
5          Calcula soma dos elementos da linha
6          Calcula novo valor x_new[i]
7
8      Calcula erro entre x_new e x_old
9      Se erro < epsilon:
10         encerra execucao
11
12     Atualiza x_old com os valores de x_new

```

O algoritmo inicia os vetores com zero (chute inicial) e para cada iteração, calcula-se os novos valores utilizando exclusivamente os dados da iteração anterior. Após o cálculo do novo vetor, é realizado o calculo do erro para verificar a convergência e no final da iteração, os valores de x_{new} são copiados para x_{old} .

Utiliza-se como critério de parada:

- Número máximo de iterações;
- Erro menor que *epsilon*.

2.2. Paralela

Na versão paralela, foi utilizado OpenMP para paralelizar os laços responsáveis pelo cálculo das aproximações, do erro e atualização dos vetores. A paralelização inicial foi aplicada no laço que é responsável pelo cálculo das aproximações x_{new} . Utilizando a diretiva (`#pragma omp parallel for schedule(static) private(soma)`)

```

1  #pragma omp parallel for schedule(static) private(soma)
2  for (int i = 0; i < n; i++) {
3      soma = 0.0;
4
5      for (int j = 0; j < n; j++) {
6          if (j != i) {
7              soma += A[i][j] * x_old[j];

```

```

8         }
9     }
10     x_new[i] = (b[i] - soma) / A[i][i];
11 }

```

Em seguida o laço do cálculo do erro também foi paralelizado utilizando *reduction* para evitar conflitos entre threads. A diretiva (*#pragma omp parallel for schedule(static) reduction(max:erro) private(diferenca)*) é usada nessa parte do paralelismo. O trecho de código que indica esse laço:

```

1  #pragma omp parallel for schedule(static) reduction(max:erro)
2  private(diferenca)
3  for (int i = 0; i < n; i++) {
4      diferenca = x_new[i] - x_old[i];
5
6      if (diferenca < 0) {
7          diferenca = -diferenca;
8      }
9      if (diferenca > erro) {
10         erro = diferenca;
11     }
12 }

```

O *reduction* permite que cada thread calcule localmente um valor máximo de erro e no final da região paralela aconteça uma combinação dos resultados automaticamente para obter o maior erro global.

Por fim no último laço, para as atualizações dos vetores é também paralelizada utilizando a diretiva (*#pragma omp parallel for schedule(static)*)

```

1  #pragma omp parallel for schedule(static)
2      for (int i = 0; i < n; i++) {
3          x_old[i] = x_new[i];
4      }

```

Como cada posição do vetor é atualizada de forma independente, não ocorre uma dependência de dados entre as iterações o que faz esse for ser paralelizado.

2.3. Decisões importantes no algoritmo paralelo

- **Número de threads:**

O programa foi executado em um processador Ryzen 7 9800X3D com 8 núcleos físicos. Com base nisso e nos resultados obtidos, para um melhor equilíbrio entre desempenho e eficiência o número ideal de threads para este algoritmo, nesse hardware é de 4 a 8. Acima disso o ganho é menos significativo, confirmando-se pelo teste para $n = 8000$, onde o speedup com 4 threads foi de 3.32x, o speedup com 8 threads foi de 3.97x e com 16 threads o speedup foi de 3.79x, onde a eficiência caiu para 23%.

- **Escalonamento (static):**

Cada iteração i executa exatamente o mesmo trabalho. Com carga uniforme entre iterações, o `schedule(static)` divide os índices em blocos contíguos iguais entre as threads antes de começar, sem overhead em tempo de execução [Yiling 2020]. O `schedule(dynamic)` é usado quando as iterações têm custo variável, o que não ocorre nesse algoritmo.

- **Variáveis privadas e compartilhadas:**

É necessário definir bem as variáveis compartilhadas e privadas em uma implementação paralela.

No nosso caso:

- As variáveis **A**, **x_old**, **b** são compartilhadas, pois são usadas apenas para a leitura, então não corre o risco de conflito.
- A variável **x_new** também é compartilhada, pois cada thread escreve em índices distintos, não causa conflito.
- A variável **soma** é privada, pois cada thread acumula seu próprio somatório, se fossem compartilhadas, as threads sobreporiam os valores umas das outras.
- A variável **diferenca** é também privada, pois cada thread calcula a sua própria diferença, se fosse compartilhada, haveria corrida na escrita.
- A variável **erro** é do tipo *reduction(max)*, pois cada thread mantém um máximo local, no final o OpenMP combina todos os máximos sem necessidade de lock.

Definir essas variáveis privadas, garante que cada thread possa manipular seus valores sem interferir nas demais.

- **Sincronização:**

A sincronização foi realizada de maneira implícita pelo próprio OpenMP ao final de cada região paralela. Essa barreira garante que todas as threads terminei o cálculos de **x_new**, que o erro seja calculado completamente e que os vetores sejam atualizados antes da próxima iteração.

Quando se quer uma barreira explícita usa-se a diretiva *barrier*, que garante que nenhuma thread pode ultrapassar o ponto onde a diretiva é definida. No algoritmo não foi necessário usar essa diretiva.

- **Paralelização**

O laço externo não foi paralelizado, pois existe uma dependência entre as iterações para o funcionamento do método. Cada iteração depende dos resultados calculados na iteração anterior, o que impossibilita a execução de forma paralela.

Portanto apenas os laços internos relevantes foram paralelizados.

3. Ambiente Experimental

Para os experimentos foi utilizado um processador Ryzen 7 9800X3D de 6000MHz com 8 núcleos físicos, 16 threads e memória RAM de 64GB DDR5 [AMD]. O sistema operacional usado foi o Windows 11 e o compilador gcc na versão 15.2.0.

3.1. Geração de matrizes

Para os testes foi implementado um gerador de matrizes para otimizar o tempo e poder testar com matrizes grandes. Essa geração passou por diversas iterações até chegar à abordagem atual.

Na versão final, os elementos fora da diagonal principal foram gerados de forma aleatória no intervalo $[1, 10]$ e o elemento da diagonal é definido como: $A[i][j] = soma_linha + soma_linha * 0.01$.

Onde $soma_linha$ é a soma de todos os elementos fora da diagonal estrita com margem mínima de 1%, forçando o método de Jacobi a realizar um grande número de iterações antes de convergir. O vetor b é gerado com os valores em um intervalo de $[1, 100]$

4. Resultados

Foram geradas matrizes aleatórias pelo gerador para os experimentos. Utilizando os critérios de parada:

- Erro menor que $\epsilon = 1E-10$;
- Máximo de iterações = 1000000

Os algoritmos foram testados em sistemas pequenos, utilizando o exemplo fornecido na especificação do trabalho e alguns exercícios complementares sobre o método iterativo de Jacobi pesquisados na internet [Ai]

4.1. Tempo de execução

Matriz 500x500

Número de iterações até convergir = 1915

Número de Threads	Tempo_seq (s)	Tempo_par (s)	Speedup	Eficiência
2	0.1810	0.2020	0.8960	44.8%
4	0.1840	0.1890	0.9735	24.3%
6	0.1770	0.2050	0.8634	14.4%
8	0.1800	0.2250	0.8000	10%
10	0.1790	0.2450	0.7306	7.3%
12	0.1790	0.2660	0.6729	5.6%
14	0.1800	0.2900	0.6207	4.4%
16	0.1770	0.3240	0.5463	3.4%

Tabela 1. Tabela de desempenho para matriz 500X500

Matriz 1000x1000

Número de iterações até convergir = 1844

Número de Threads	Tempo_seq (s)	Tempo_par (s)	Speedup	Eficiência
2	0.7080	0.4530	1.5629	78.1%
4	0.7060	0.3060	2.3072	57.7%
6	0.7060	0.2850	2.4772	41.3%
8	0.6980	0.2830	2.4664	30.8%
10	0.7010	0.3030	2.3135	23.1%
12	0.6960	0.3110	2.2379	18.6%
14	0.6990	0.3420	2.0439	14.6%
16	0.7060	0.3510	2.0114	12.6%

Tabela 2. Tabela de desempenho para matriz 1000X1000

Matriz 2000x2000

Número de iterações até convergir = 1775

Número de Threads	Tempo_seq (s)	Tempo_par (s)	Speedup	Eficiência
2	2.7550	1.4860	1.8540	92.7%
4	2.7430	0.8310	3.3008	82.5%
6	2.7600	0.6340	4.3533	72.6%
8	2.7410	0.5620	4.8772	61%
10	2.7580	0.5780	4.7716	47.7%
12	2.7560	0.5470	5.0384	42%
14	2.7400	0.5170	5.2998	37.9%
16	2.7410	0.5480	5.0018	31.3%

Tabela 3. Tabela de desempenho para matriz 2000x2000

Matriz 4000x4000

Número de iterações até convergir = 1706

Número de Threads	Tempo_seq (s)	Tempo_par (s)	Speedup	Eficiência
2	12.2670	6.2870	1.9512	97.6%
4	12.2290	3.2780	3.7306	93.3%
6	12.4100	2.6050	4.7639	79.4%
8	12.1810	2.1080	5.7785	72.2%
10	12.1000	1.8750	6.4533	64.5%
12	12.3430	1.9430	6.3525	52.9%
14	12.4730	1.7270	7.2224	51.6%
16	12.3770	1.9930	6.2102	38.8%

Tabela 4. Tabela de desempenho para matriz 4000x4000

Matriz 8000x8000

Número de iterações até convergir = 1634

Número de Threads	Tempo_seq (s)	Tempo_par (s)	Speedup	Eficiência
2	48.2720	28.1260	1.7163	85.8%
4	48.2720	14.5370	3.3206	83.0%
6	48.2840	12.4160	3.8889	64.8%
8	48.3230	12.1640	3.9726	49.7%
10	48.7220	12.2400	3.9806	39.8%
12	48.7980	12.2600	3.9803	33.2%
14	48.6580	12.3200	3.9495	28.2%
16	48.7730	12.8430	3.7976	23.7%

Tabela 5. Tabela de desempenho para matriz 8000x8000

Para método de comparação, abaixo estão as tabelas de tempo de execução dos exercícios e do exemplo dado na descrição do trabalho

Teste exemplo

Número de iterações até convergir = 11

Número de Threads	Tempo_seq (s)	Tempo_par (s)	Speedup	Eficiência
2	0.0000	0.0020	0.0000	0%

Tabela 6. Tabela de desempenho do exemplo

Teste exercício 1

Número de iterações até convergir = 31

Número de Threads	Tempo_seq (s)	Tempo_par (s)	Speedup	Eficiência
2	0.0000	0.0020	0.0000	0%

Tabela 7. Tabela de desempenho do exercício 1

Teste exercício 2a

Número de iterações até convergir = 28

Número de Threads	Tempo_seq (s)	Tempo_par (s)	Speedup	Eficiência
2	0.0000	0.0020	0.0000	0%

Tabela 8. Tabela de desempenho para o exercício 2a

Teste exercício 5

Número de iterações até convergir = 29

Número de Threads	Tempo_seq (s)	Tempo_par (s)	Speedup	Eficiência
2	0.0000	0.0060	0.0000	0%

Tabela 9. Tabela de desempenho para o exercício 3a

Os resultados obtidos com esses testes, reforçam a conclusão que para sistemas pequenos, paralelizar não traz ganhos significativos e que geralmente o algoritmo sequencial acaba sendo mais eficaz

4.2. Convergência

Optamos por mostrar as matrizes de teste em vez do número de threads para cada matriz na tabela de convergência, pois os resultados mostram que não há relação entre o número de threads e a quantidade de iterações até a convergência.

Matrizes de teste	Iterações até convergir	Erro final	Tolerância
500X500	1915	9.92e-11	10^{-10}
1000X1000	1844	9.94e-11	10^{-10}
2000X2000	1775	1.00e-10	10^{-10}
4000X4000	1706	9.92e-11	10^{-10}
8000X8000	1634	9.91e-11	10^{-10}
exemplo	11	9.00e-10	11^{-10}
exercício 1	31	5.15e-11	10^{-10}
exercício 2a	28	5.15e-11	10^{-10}
exercício 5	77	7.99e-11	10^{-10}

Tabela 10. Tabela de convergência do método para matrizes geradas

Abaixo estão os gráficos da convergência para alguns dos testes

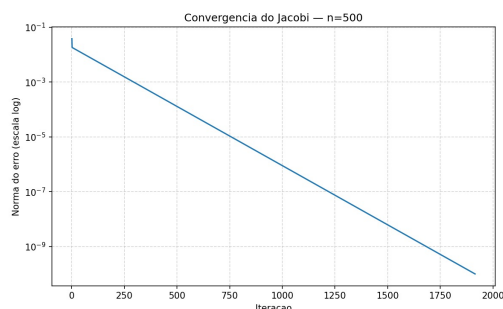


Figura 1. Gráfico da convergência para a matriz 500X500

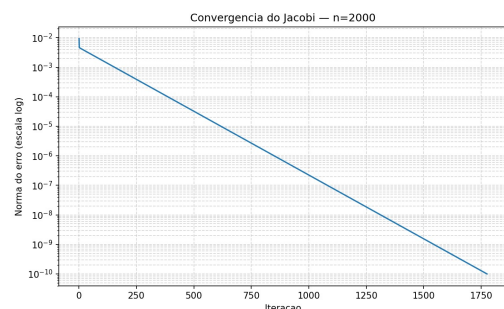


Figura 2. Gráfico da convergência para a matriz 2000X2000

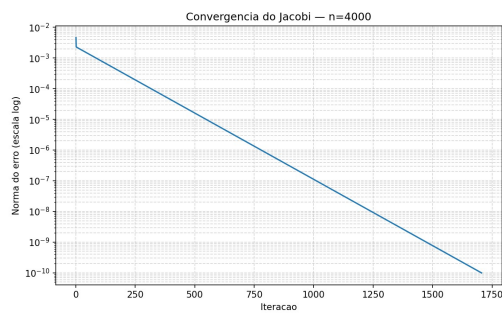


Figura 3. Gráfico da convergência para a matriz 4000X4000

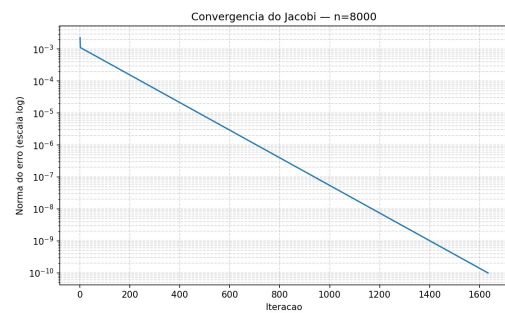


Figura 4. Gráfico da convergência para a matriz 8000X8000

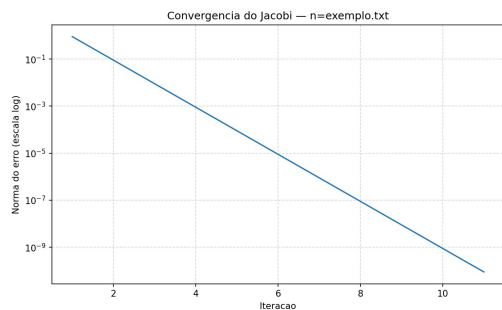


Figura 5. Gráfico da convergência para a matriz do exemplo

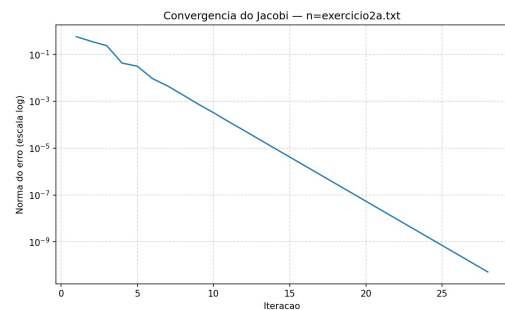


Figura 6. Gráfico da convergência para a matriz do exercício 2a

4.3. Discussão

- **Impacto dos número de threads no desempenho**

Com os resultados obtidos, observou-se que com o aumento de threads reduziu significativamente o tempo de execução em sistema de grande dimensão. Nota-se que a matriz 500x500 não obteve essa grande redução do sequencial para o paralelo. Para matrizes maiores, como a 4000x4000 e a 8000x8000, ocorreu um crescimento expressivo no speedup conforme o aumento das threads. Na matriz 4000x4000, por exemplo, a execução com 4 threads alcançou um speedup de 3.73x, com 8 threads alcançou 5.77x e com 12 threads alcançou 6.35x. Entretanto nota-se que ao usar 16 threads o speedup caiu para 6.21x, evidenciando uma saturação no ganho de desempenho.

Com isso, os experimentos indicam que com base do hardware usado a faixa de 4 a 8 threads são suficiente para obtermos o melhor equilíbrio entre desempenho e eficiência. Acima disso os ganhos não são significativos e tornam-se menores.

- **Limitações encontradas**

Uma limitação é dada em matrizes pequenas como a nossa 500x500, devido ao overhead associado ao gerenciamento de threads, o custo da criação e sincronização das threads é superior ao ganhos obtidos com o paralelismo. Nesse caso, a versão paralela

mostrou desempenho inferior ao da sequencial. Por exemplo, utilizando 8 threads, o speedup obtido foi 0.80x, o que indica um pior desempenho em comparação com o algoritmo sequencial.

A solução apresentada possui uma limitação quanto a escalabilidade, pois a eficiência cai progressivamente à medida que o número de threads aumenta. Apesar do tempo de execução continuar caindo em alguns casos, os ganhos tornam-se desprezíveis conforme o número de threads aumenta para um tamanho de matriz fixo.

- **Comparação entre sistemas pequenos e grandes**

Com os resultados dos testes, é possível observar uma grande diferença do algoritmo para sistemas pequenos e grandes.

Em sistemas menores como o de 500x500, o tempo de execução no algoritmo sequencial já era relativamente baixo, na versão paralela não foi obtido um ganho muito grande. Já nas matrizes maiores como a 8000x8000, o tempo de execução aumentou bastante no sequencial devido à maior quantidade de cálculos que são necessários para realizar cada iteração do método. Portanto nesses casos, a versão paralela consegue reduzir de forma significativa o tempo de execução.

Notou-se também que a medida que o tamanho da matriz aumentava (matrizes geradas pelo nosso gerador), o método precisava de menos iterações para convergir.

5. Conclusão

Nesse trabalho foi realizada a implementação do método de Jacobi nas versões sequencial e paralelizada com OpenMP, com objetivo de analisar o impacto que o paralelismo causa no desempenho do algoritmo.

Os resultados mostraram que a implementação paralela preservou o comportamento matemático do método de Jacobi e produziu as mesmas soluções, quantidade de iterações e erro final que o algoritmo sequencial. Nos testes com sistemas pequenos, observou-se que o uso de OpenMP não trouxe ganhos de desempenho, já que o custo de criação e sincronização das threads acabou sendo maior que o próprio custo computacional do problema. No entanto, para matrizes grandes a versão paralela apresentou redução significativa no tempo de execução, alcançando speedups consideráveis e demonstrando melhor aproveitamento do paralelismo. Além disso, foi possível notar que o aumento do número de threads melhora o desempenho apenas até determinado ponto.

De forma geral, o trabalho permitiu compreender na prática conceitos importantes de programação paralela, como paralelismo de dados, speedup, eficiência, sincronização e overhead.

6. Referências

Seção para acrescentar referência que não são citadas ao longo do texto, mas que foram usadas para realização do trabalho.

[School], [Volmir Eugênio Wilhelm 2018], [OpenMp]

Referências

Ai, R. **Exercícios de Método de Gauss-Jacobi - Lista de Exercícios Resolvida**. <http://www.respondeai.com.br/conteudo/calculo-numerico/sist>

emas-lineares-e-nao-lineares/lista-de-exercicios/metodo-de-gauss-jacobi-1196.

AMD. **Processador para desktop AMD Ryzen™ 7 9800X3D**. <https://www.amd.com/pt/products/processors/desktops/ryzen/9000-series/amd-ryzen-7-9800x3d.html>.

Boccardo, M. E. (2017). **Sistemas Lineares: Aplicações e propostas de aula usando a metodologia de resolução de problemas e o software geogebra**. Disponível em: <https://repositorio.unesp.br/server/api/core/bitstreams/24cb4fc7-87ca-4928-8521-4a7a51aab70a/content>.

OpenMp. **Who's Using OpenMP?** <https://www.openmp.org/about/whos-using-openmp/>.

Romanazzi, G. (2020). **Método iterativos para a resolução de sistemas lineares. Método de Jacobi**. <https://www.ime.unicamp.br/~roman/courses/MS211/1s2020/metodositerativos1.pdf>.

School, S. P. **OpenMP Parallel Programming Full Course: 5 Hours**. <https://www.youtube.com/watch?v=FjCFT5ojQSk>.

Volmir Eugênio Wilhelm, M. K. (2018). **Métodos Numéricos Sistemas Lineares – Métodos Iterativos**. https://docs.ufpr.br/~volmir/MN_10_resolucao_sistemas_metodos_iterativos_ppt.pdf.

Yiling (2020). **OpenMP - Scheduling (static, dynamic, guided, runtime, auto)**. Disponível em: <https://610yilingliu.github.io/2020/07/15/ScheduleinOpenMP/>.